# The Relay — submission

Every table number regenerates from `outputs/results.csv` (`scripts/make_tables.py`); see README.

## 1 What I built and why

**Pipeline:** admission (Groq-mirroring token buckets, FIFO queue, 20 s patience) → analyse (local keyword routing) → **retrieve** (tool call `search_papers`, BM25 over page-tagged chunks) → evidence gate (weak → expand once, else refuse without a model call) → draft (Groq `openai/gpt-oss-20b`, streamed) → verify (local: every `[paper p.N]` must be a retrieved page, every multi-digit number must appear in a cited passage; fail → one revise, else refuse). **The decision that shaped it most: tokens, not requests or latency, are the scarce resource.** An answer costs ~3.3K tokens, so 8,000 TPM allows ~2.4 answers/min (RPM would allow 30) and the daily token limit ~60 answers. Hence one model call per answer, deterministic analysis and verification, and under pressure a cheaper retrieval setting (A) instead of turning people away. "Fast" = first token ≤ 1 s at p95 when not overloaded; nobody waits over 20 s.

## 2 The real service and my stand-in

Found by hitting each limit (`scripts/calibrate_groq.py`, 80 requests; TPD by really exhausting the day) and reading headers and 429 bodies (`outputs/calibration.jsonl`).

| Limit | Value you found | How you found it | Confidence |
|---|---|---|---|
| Requests per minute | 30 | 36 concurrent → 8 × 429 "RPM: Limit 30" | high |
| Tokens per minute | 8,000 | header; 429 "Limit 8000, Used 6389, Requested 2177" (= prompt + max tokens) | high |
| Requests per day | 1,000 | header; `remaining` 966 → 937 over 36 calls, 28 ok: 429s not counted | high |
| Tokens per day | 200,000 | real 429 "Limit 200000, Used 199792 … try again in 36m30s" | high |
| Requests in flight at once | ≥ 12, none refused for concurrency | 12 concurrent: all 200, 388 → 2,375 ms (queued) | medium |
| What it returns when a limit is hit | 429 + `retry-after` + JSON naming limit, used, requested, wait | captured for all three | high |
| Time until it recovers | RPM 2.8 s; TPM 3.8 s; TPD 36.5 min for a 5K request | polled to first 200; TPD `retry-after` | high; TPD medium |

**Finding:** every limit is a continuously refilling bucket, not a counter that resets. TPM refills at 133 tokens/s and TPD at 2.3 tokens/s, and each `retry-after` equals the deficit divided by the refill rate.

**Stand-in** (`src/relay/standin_server.py`): a Groq-compatible SSE server (same client, different `base_url`) enforcing the measured buckets with Groq-shaped 429s. Calibrated: first token lognormal (median 420 / p95 800 ms; real p50 404/519 ms) + 58 ms per 1K prompt tokens (Groq `prompt_time`); ~1 token/chunk; 1.17× slower per extra request in flight above 2 (12-concurrent probe). It replays the recorded real answer per question and setting, so its rows carry real grades. Synthetic faults, never seen on Groq: 1% 5xx, 3% slow successes.
**Where it differs most:** Groq's `queue_time` is 60–70% of its first-token time and depends on other tenants; mine only on my own traffic. Real 429s took 2–3 s under concurrency; mine are instant. Unmeasured: I assume unused `max_completion_tokens` are refunded to the TPM bucket, and Table 2's overload capacity depends on it.

## 3 Results

Real: `real_steady`, `real_daily_exhausted` and Table 3. All else: 15-minute open-loop stand-in runs. p50/p95/p99, n in brackets. First token = the model's first token; the person sees the reply only after verification (`released_ms` ≈ total), the price of never sending an unchecked answer. First token covers `ok` rows, total covers all rows, so instant 429s put `overload_retry_off`'s total p50 below its first-token p50.

| Scenario | Real or stand-in | Requests sent | Duration (s) | Offered rate per min | First token ms p50 / p95 / p99 | Total time ms p50 / p95 / p99 |
|---|---|---|---|---|---|---|
| steady | Stand-in | 21 | 895.0 | 1.4 | 604 / 830 / 1,043 (n=21) | 640 / 934 / 1,120 (n=21) |
| burst | Stand-in | 30 | 896.3 | 2.0 | 615 / 10,142 / 17,424 (n=14) | 20,002 / 20,003 / 20,004 (n=30) |
| sustained_overload | Stand-in | 84 | 905.8 | 5.7 | 13,717 / 19,090 / 19,415 (n=73) | 14,759 / 20,003 / 20,004 (n=84) |
| rpm_halved | Stand-in | 21 | 914.4 | 1.4 | 3,984 / 13,917 / 14,454 (n=16) | 5,980 / 20,005 / 20,006 (n=21) |
| daily_exhausted | Stand-in | 30 | 290.0 | 6.2 | not measured | 2 / 3 / 3 (n=30) |
| real_steady | Real | 10 | 270.7 | 2.2 | 663 / 721 / 735 (n=10) | 714 / 775 / 798 (n=10) |
| overload_retry_on | Stand-in | 84 | 892.8 | 5.7 | 9,739 / 18,583 / 18,924 (n=57) | 9,798 / 18,868 / 19,430 (n=84) |
| overload_retry_off | Stand-in | 84 | 891.6 | 5.7 | 642 / 1,758 / 3,598 (n=43) | 391 / 1,158 / 3,151 (n=84) |
| real_daily_exhausted | Real | 3 | 40.0 | 4.5 | not measured | 2 / 93 / 101 (n=3) |

| Scenario | Real or stand-in | Succeeded % | Rate-limited % | Timed out or given up % | Other errors % | Retries per request (mean) | Peak queue length | Tokens per answer (mean) | Cost per 1,000 answers |
|---|---|---|---|---|---|---|---|---|---|
| steady | Stand-in | 100.0% | 0.0% | 0.0% | 0.0% | 0.00 | 0 | 3,192 | $0 (free tier); paid: not measured |
| burst | Stand-in | 46.7% | 0.0% | 53.3% | 0.0% | 0.00 | 17 | 2,945 | $0 (free tier); paid: not measured |
| sustained_overload | Stand-in | 86.9% | 0.0% | 13.1% | 0.0% | 0.01 | 1 | 1,559 | $0 (free tier); paid: not measured |
| rpm_halved | Stand-in | 76.2% | 4.8% | 19.0% | 0.0% | 0.05 | 0 | 3,096 | $0 (free tier); paid: not measured |
| daily_exhausted | Stand-in | 0.0% | 100.0% | 0.0% | 0.0% | 0.00 | 0 | not measured | $0 (free tier); paid: not measured |
| real_steady | Real | 100.0% | 0.0% | 0.0% | 0.0% | 0.00 | 0 | 3,107 | $0 (free tier); paid: not measured |
| overload_retry_on | Stand-in | 67.9% | 31.0% | 1.2% | 0.0% | 0.40 | 1 | 2,130 | $0 (free tier); paid: not measured |
| overload_retry_off | Stand-in | 51.2% | 48.8% | 0.0% | 0.0% | 0.00 | 0 | 2,910 | $0 (free tier); paid: not measured |
| real_daily_exhausted | Real | 0.0% | 100.0% | 0.0% | 0.0% | 0.00 | 0 | not measured | $0 (free tier); paid: not measured |

**Not guessable without running it:** under sustained overload (5.7 offered/min vs ~2.4/min capacity) **86.9% still succeeded**, because admission degraded 81 of 84 requests to setting A (1,559 vs 3,192 tokens per answer), but the median person waited 14.8 s. In the burst the queue reached 17 and 16 of 30 gave up at 20 s, all still in our queue.

## 4 Where the time goes

Slowest real request (`6e2f7c76314f`, q11, setting C): **12,037 ms = 10,408 waiting + 1,628 working + 0 retrying**, and the delay came **after** the first token (820 ms). The draft finished at 889 ms but failed verification: it cited `[paper_id p.10]`, copying our prompt's placeholder, and gave BPIC 6.0 (the heuristic baseline's figure, not 4.8). The revise call needed a 3.5K-token reservation and waited 10.4 s in our own admission queue (`queue_enter` → `queue_exit`), then returned the same text and was refused. Every phase boundary is timestamped in `events.jsonl`; `scripts/slow_request.py` prints the split.

### 5 When a limit is hit

**Per-minute:** requests wait FIFO for the bucket to refill. Below 30% TPM headroom new requests get setting A (weaker: 64.7% vs 76.5% correct). Whoever would exceed 20 s in the queue is dropped (`gave_up_in_queue`), so in a burst the latest arrivals lose. A provider 429 pauses all admissions for its `retry-after`. **Per-day:** nothing is queued or retried. The first TPD 429, or our guard at 90% of TPD, marks the day exhausted and everyone after gets the fixed reply in ~2 ms. In the real run Groq's 429 took 103 ms and the next two were refused locally in 1–2 ms. Our guard is a fixed counter, so it fired early (~180K tokens) and blocked 4 evaluation rows (I re-ran one and dropped q18). I reset it deliberately for `real_steady` to observe Groq's own response; Groq accepted ~20K more tokens because its bucket refills.

### Retrieval: quality versus cost

| Retrieval setting | Real or stand-in | Questions asked | Graded (n) | Correct and cited % | Wrong % | Correctly refused % | Wrongly refused % | Tokens per answer (mean) | First token ms p50 / p95 | Total time ms p50 / p95 |
|---|---|---|---|---|---|---|---|---|---|---|
| A:250w/k3/rerank=no | Real | 18 | 17 | 64.7% | 11.8% | 11.8% | 11.8% | 1,655 | 591 / 1,787 (n=17) | 603 / 1,792 (n=18) |
| B:450w/k4/rerank=no | Real | 18 | 17 | 76.5% | 5.9% | 11.8% | 5.9% | 3,327 | 751 / 2,453 (n=17) | 806 / 4,097 (n=18) |
| C:450w/k4/rerank=yes | Real | 18 (19 rows) | 17 | 70.6% | 11.8% | 11.8% | 5.9% | 3,387 | 767 / 5,191 (n=17) | 791 / 11,591 (n=19) |

Real runs, paced 25 s apart. Grade shares are of 17 graded rows per setting (15 answerable, 2 unanswerable), so 11.8% correctly refused = both unanswerables refused. **I chose B:** most correct-and-cited (76.5%) and fewest wrong (5.9%). C's re-ranking mostly reordered the same chunks (its slow p95 is two requests queued behind a revise call). A halves the tokens but loses 12 points of correctness. **Near the limit** the system switches to A: a cheaper cited answer beats no answer.

### How this library will mislead a naive retriever

1. **Different words for the same thing.** "What *practical* latency does Moshi report?" Page 1 says "200ms in *practice*". BM25 missed it, so all three settings answered "not reported".
2. **Symbols.** "Adam beta1, beta2, epsilon?" The paper writes β1/β2/ϵ, so the best passage scored 0.36. My first gate (0.50) refused it before asking the model; I lowered the gate to 0.30.
3. **One word, several meanings; one result, two numbers.** "Transformer" appears in all three papers: on q15, one answer cited H-Net for a claim about the original Transformer. Attention v7 reports 41.8 BLEU in its abstract and Table 2 but 41.0 in §6.1. One real answer said "41.0 in Table 2" (`outputs/archive/adhoc_real/q02_A_rerun`).

### 6 Retries

**Policy:** at most one retry, only on 429, timeout or 5xx; wait for `retry-after`, else full-jitter backoff (0.5–8 s); never on a daily limit, after tokens have streamed, or past the person's patience. SDK retries are off, so there is one retry layer. **Where retrying made things worse** (stand-in, client admission opened 10×, same 84 arrivals): retries raised completions from 43 to 57, but provider 429s rose from 41 to 58, because retried requests arrive just as the bucket refills. Each 429 restarts our global cool-down, so 34 requests waited over 5 s in our own queue against 0 without retries. `d4a9ed359c28` (q07) never retried, arrived to an empty queue and waited 10.3 s. Median first token went from 642 ms to 9,739 ms, and 15 requests that succeeded without retries failed with them. On real data, the revise in heading 4 doubled one request's tokens for the same answer.

### 7 What the person sees

At 3 pm, within ~2 ms: *"I can't answer right now: today's usage allowance for the language-model service has been used up. Daily allowances reset once a day, so please try again after the reset. I have not guessed an answer."* My own measurement shows this is wrong: the bucket refills continuously, so it should say "try again in about N minutes", with N from `retry-after` (36 min in my run). That is my next change.

### 8 What I'm unsure about, what I left out, what I learned

**Least certain assumption:** that Groq's free-tier queueing stays near 300 ms under launch-day load from other tenants. If it grows, the stand-in's overload latencies are optimistic and 20 s patience drops more people than Table 2b shows. **Deliberately not built:** an embedding retriever (stemming would fix "practice/practical" without another model) and an answer cache (it saves tokens but hides the behaviour this brief asks me to measure). Three bugs found in the measured runs are listed in the README and left unfixed, so the code matches the numbers. **New to me:** [YOU: in your own words.]

### Your feedback on Vaani

[YOU: visit https://www.vaani.dev/: what stood out, what confused you, one thing you'd change.]

### How I worked with the assistant

Claude Code orchestrating (Opus), Sonnet sub-agents building modules against a shared contract (`SPEC.md`, `src/relay/types.py`), Haiku for docs research and candidate questions, an Opus advisor reviewing plan and results. Unedited history: `report/conversation_history.md` (raw JSONL in `report/raw_transcripts/`). [YOU, half a page: the three prompts that mattered most and why; one thing it got wrong that **you** caught and how; one decision where **you** overruled it; what is distinctly yours.]